{ print("1 1 1"); → 1 1 1 → 1 1 1 → 1 1 1 } 100

↓
×3

100 times

simplest way:

nested loop //

① → [1] → 100 times

automated?

for(i=1; i<=100; i++){

↳ for(j=1; j<=100; j++)

print("1 ");

}}

$n=4$

$i=1 \rightarrow 1$

$i=2 \rightarrow 1\ 2$

$i=3 \rightarrow 1\ 2\ 3$

$i=4 \rightarrow 1\ 2\ 3\ 4$

code

$n=4$ ← input

1
1 2
1 2 3
1 2 3 4

$i \rightarrow$ 1 .... (i)

1 2 3 .... (i)

$n=3$

1
1 2
1 2 3

for(i=1; i<=n; i++) {

n → changes

line number

for(j=1; j<=i; j++) {

content

System.out.print(j+" ");

}

System.out.println();

i=2; ② j=1

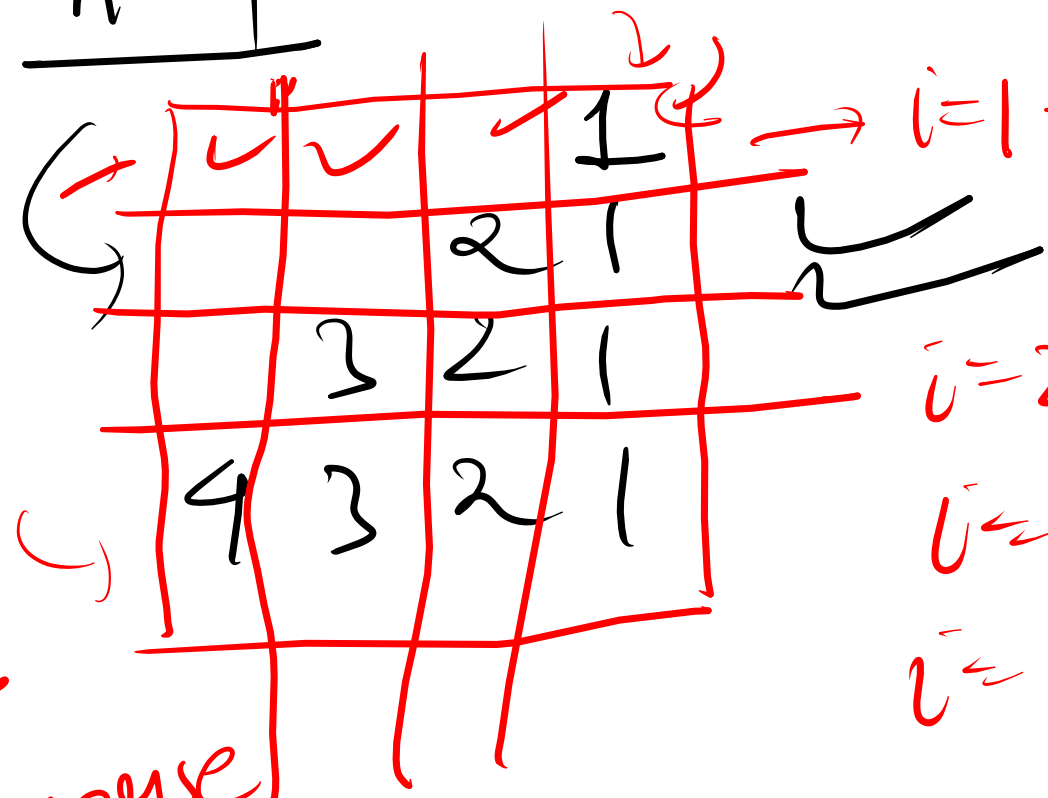
n=3

1

2 1

3 2 1

n=4



i → 3 spaces → 1

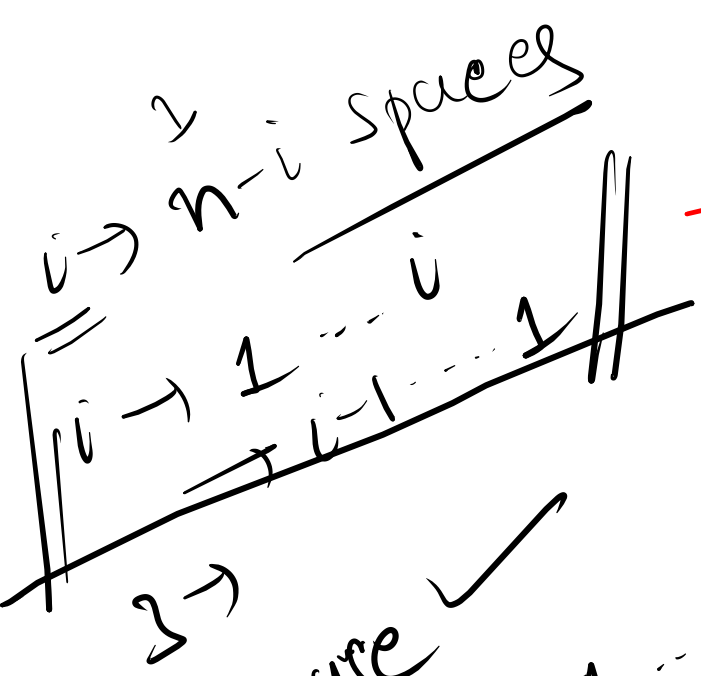
i=2 → — — 2 1

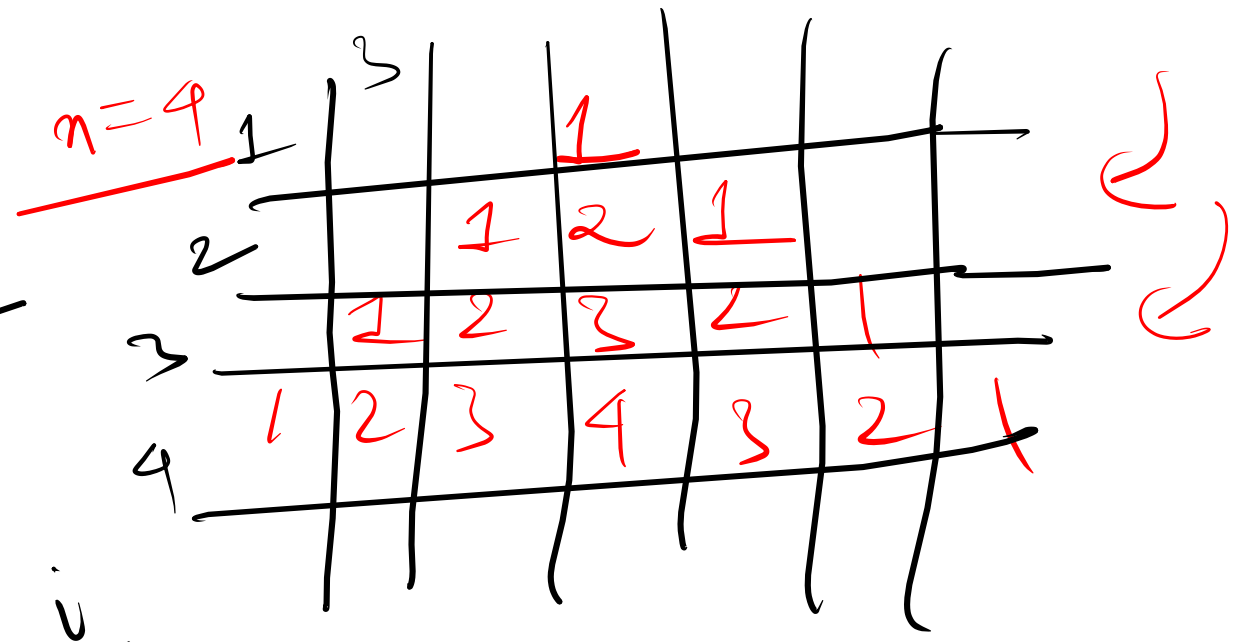
i=3 — 3 2 1

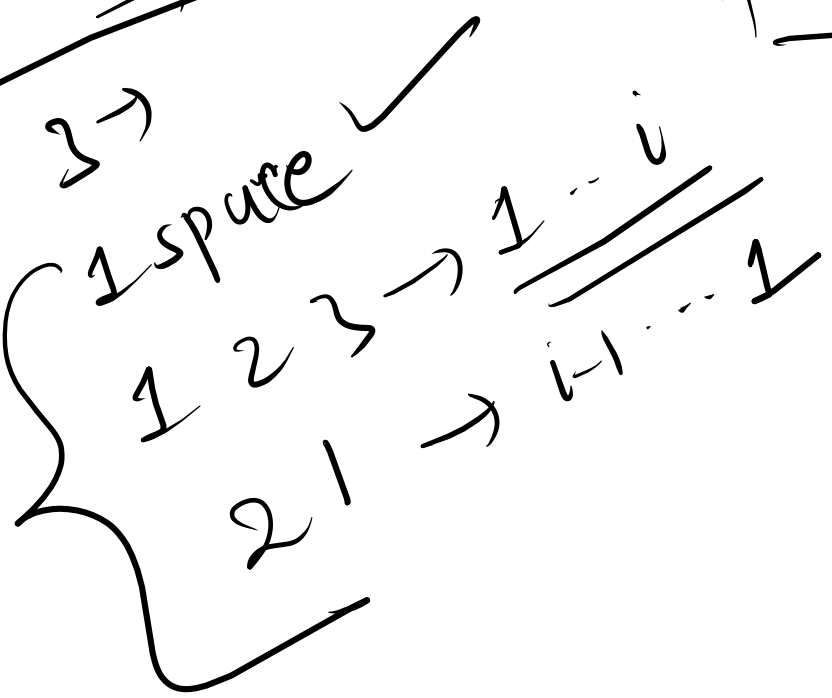
i=4 → 4 3 2 1

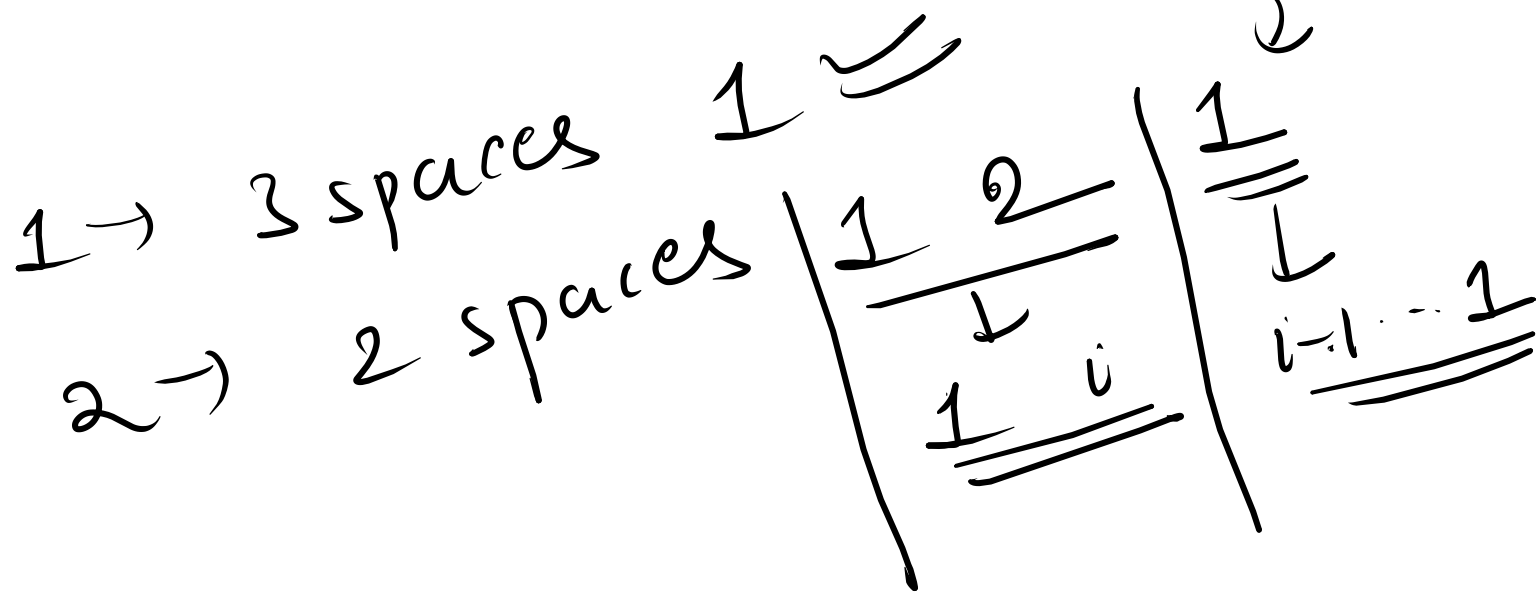
i → n-i spaces

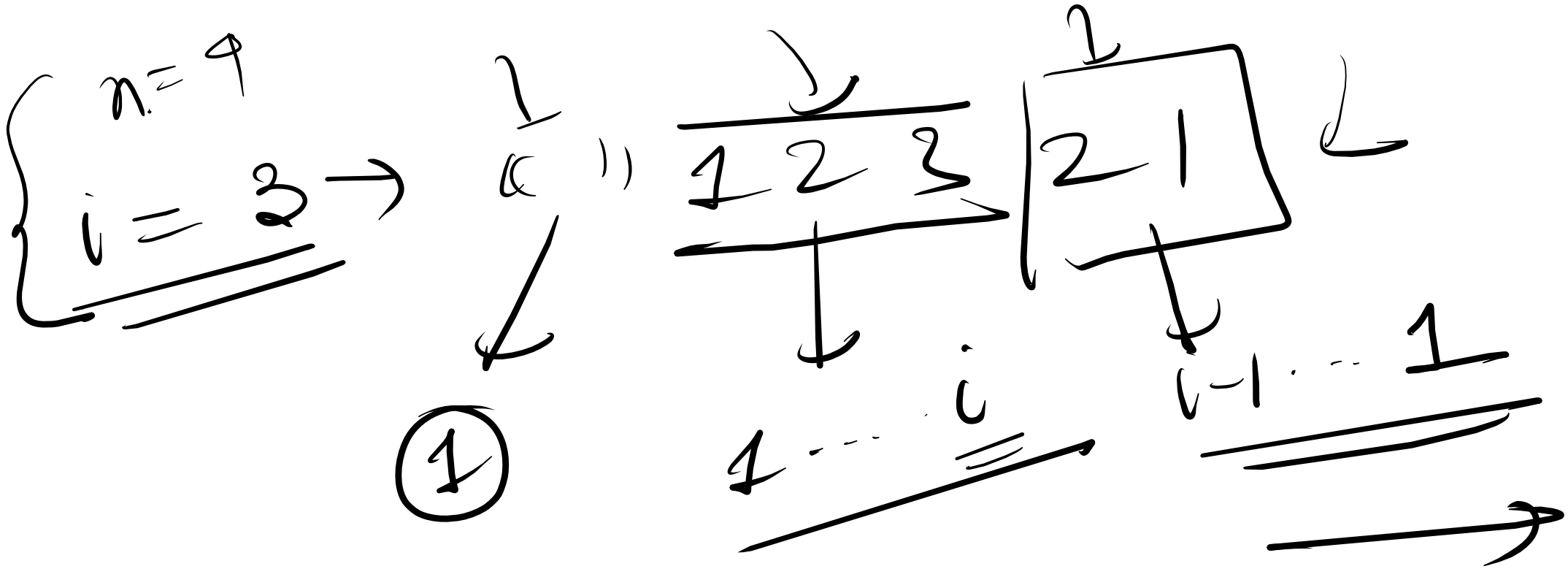
i ... 1 → reverse order

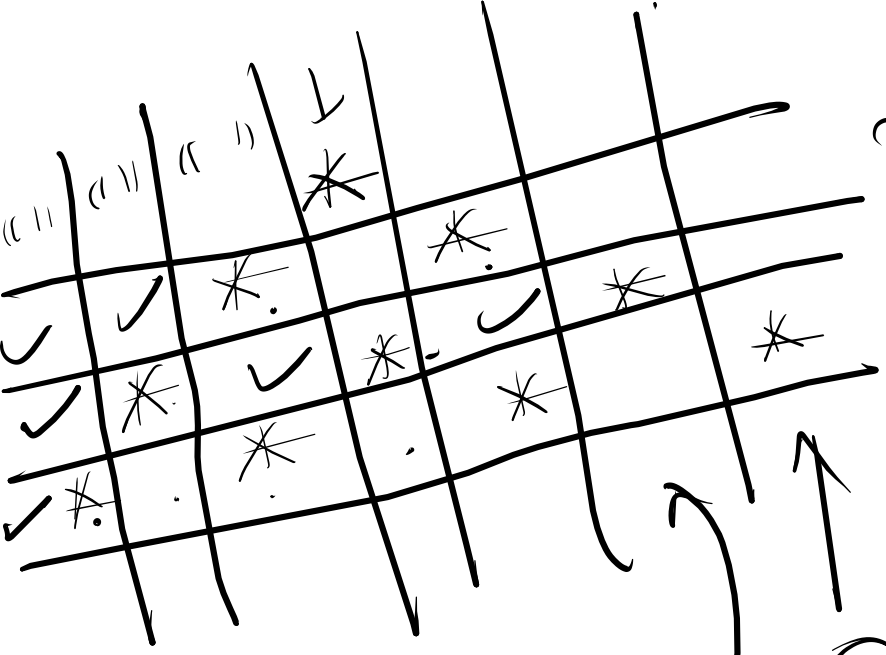

i → n-i spaces
i → 1 ... i
i-1 ... 1
3 →


n=4
1
2
3
4
1
1 2 1
1 2 3 2 1
1 2 3 4 3 2 1


1 space
1 2 3 → 1 ... i
2 1 → i-1 ... 1
2


1 → 3 spaces
2 → 2 spaces
1
1 2
1
1 i
1
i-1 ... 1


n = 4
i = 3
1
1 2 3
2 1
1 ... i
i-1 ... 1
1

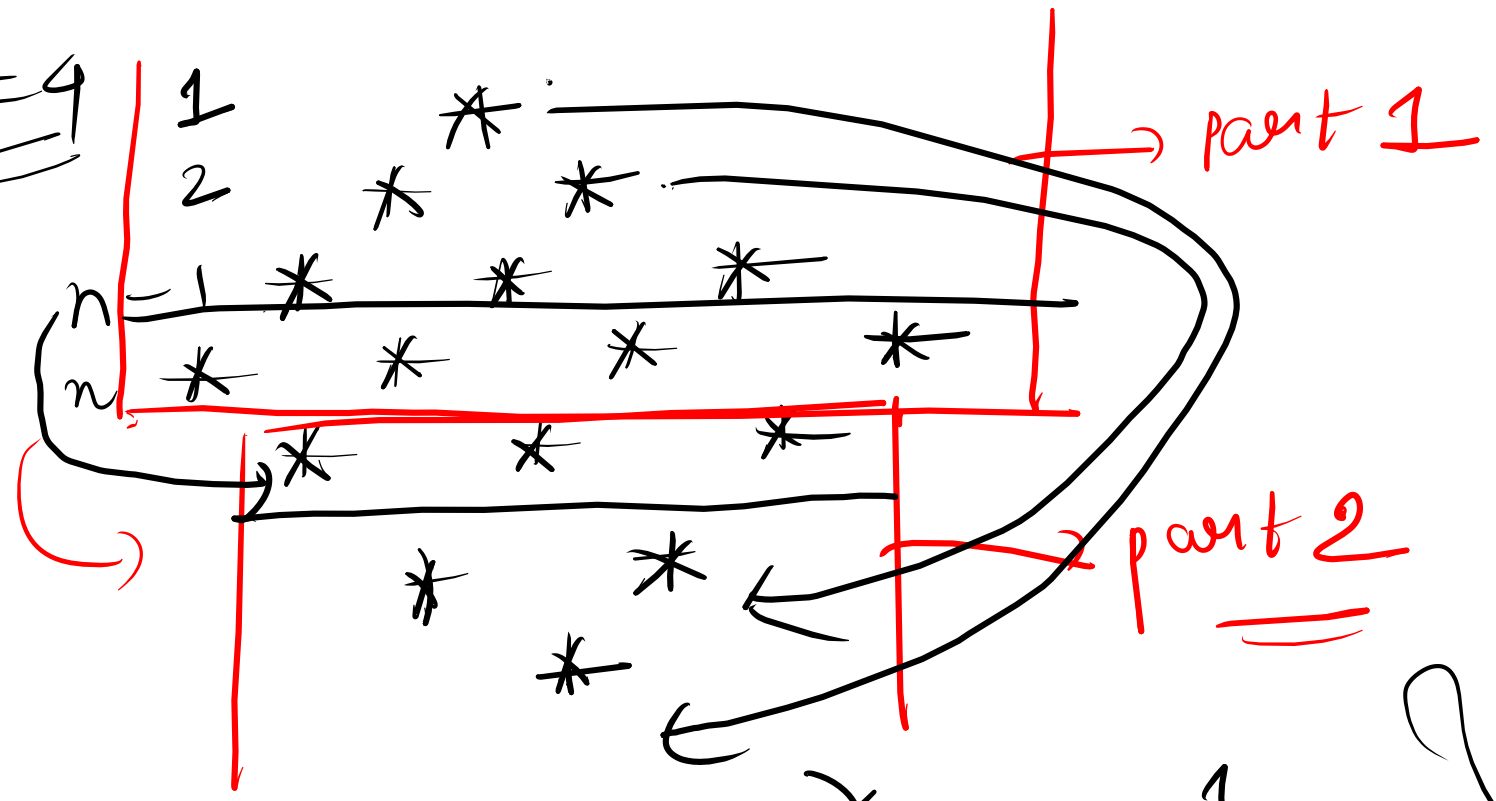


n = 4

n

4

1 = n - 1 spaces



i stars

part 2 →

n-1 → 1

n-2 → 2

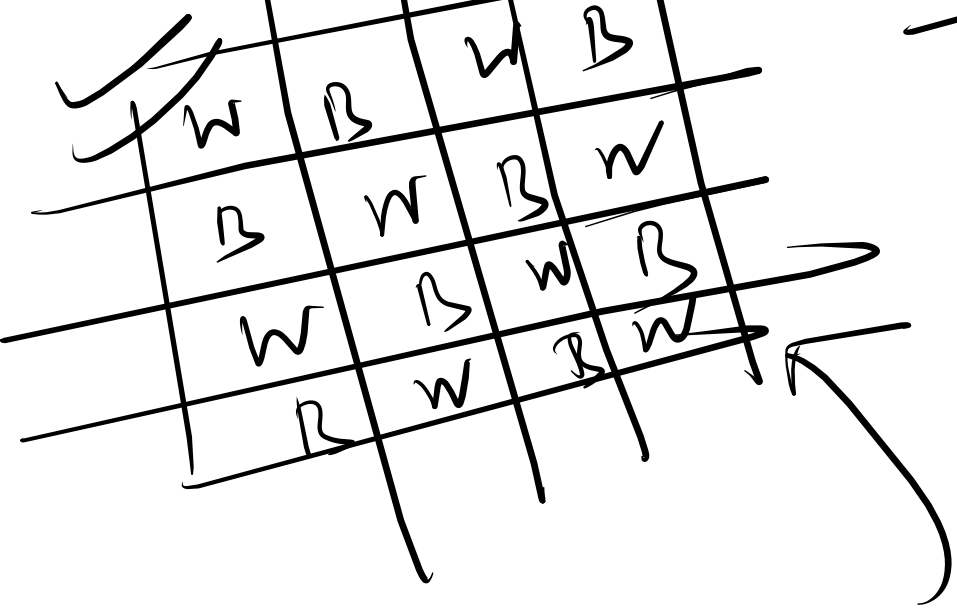
n-3 → 3

1 → n-1

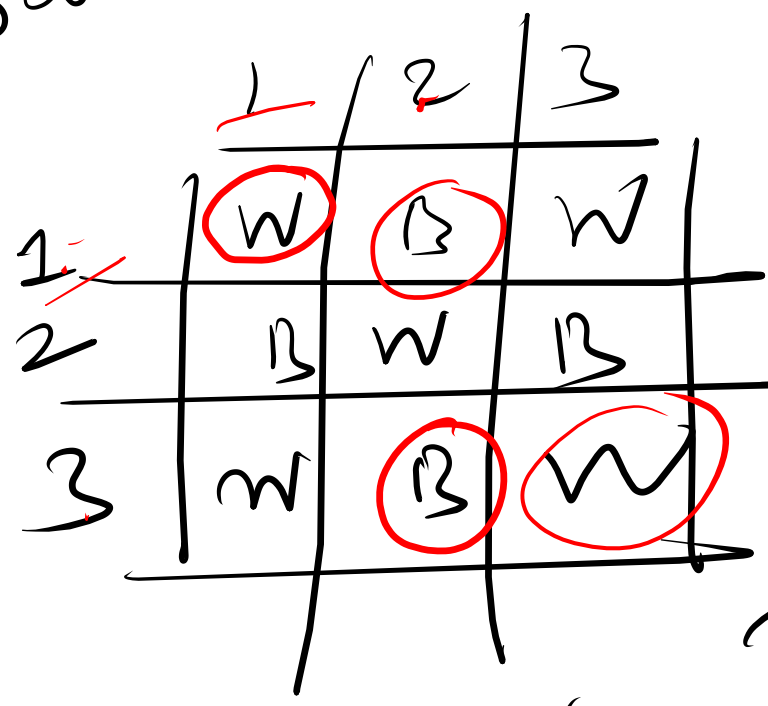
n*n

chess Board

n = 4



n = 3



(r+c) % 2 == 0

↳ even

W

↳ odd

B

5 * 4

num = 1

n = 5 → rows
m = 4 → cols.

1 2 3 4
5 6 7 8
9 0 1 2
3 4 5 6
7 8 9 0

5 * 4

num / 10

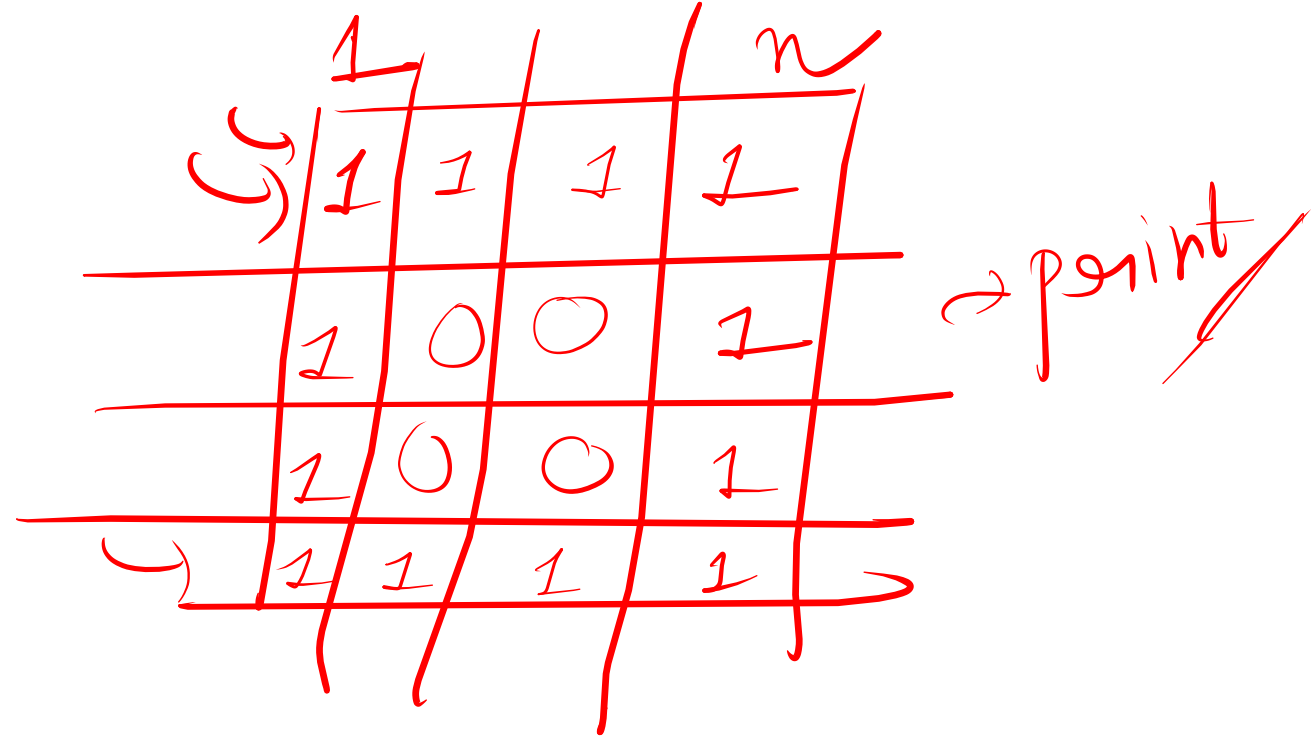
1 2 3 4
5 6 7 8
9 0 1 2
3

12
13

11 → 1
2 → 2
1 → 3

n x n

8 * 4



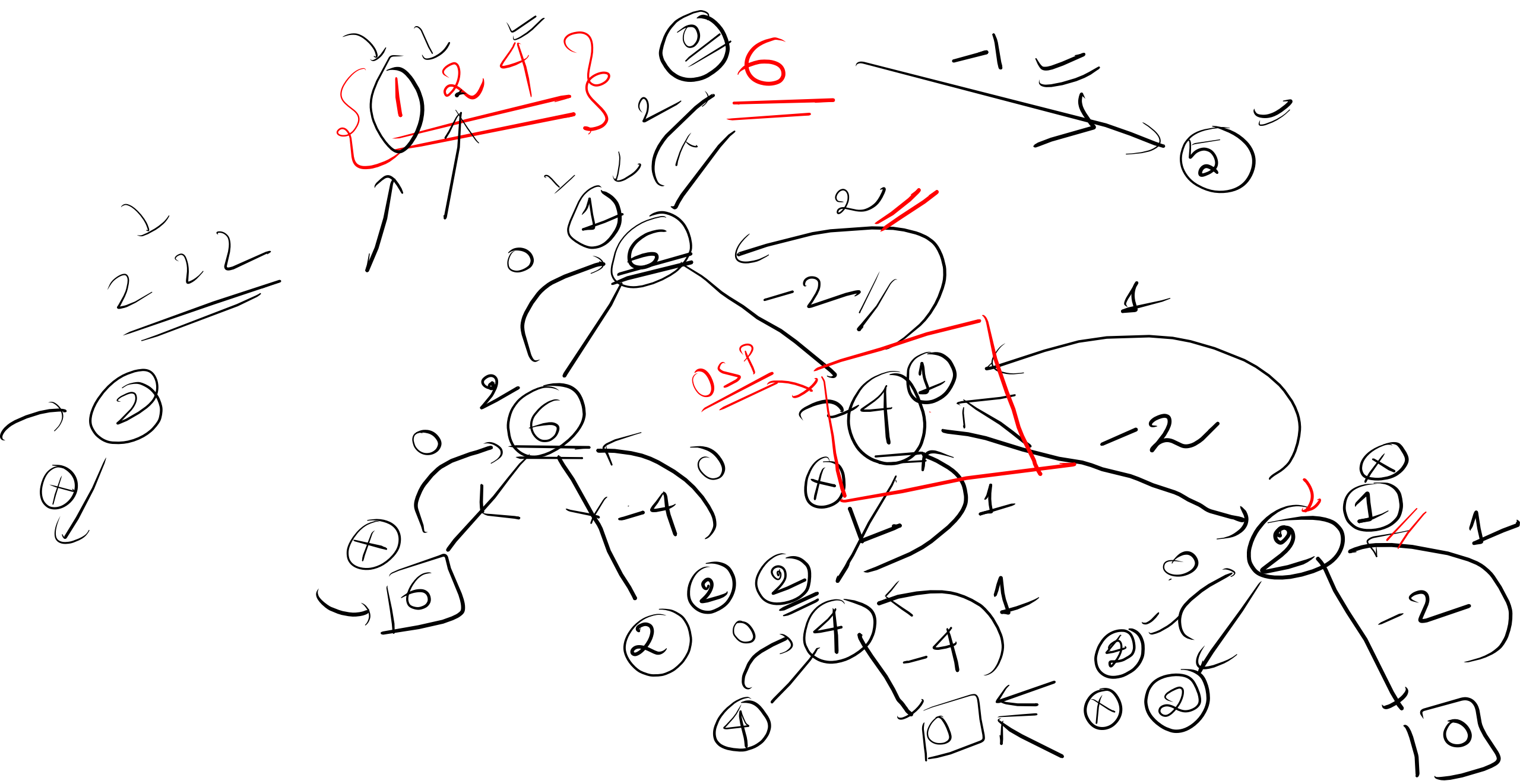
i/j

```
for(i=1/n
  for(j=1/n
```

i/j

```
for(i=1/n
```

```
for(i=1; i<=n; i++){
    for(j=1; j<=n; j++){
        if(i==1 || i==n ||
           j==1 || j==n)
            print('0');
        else
            print('1');
    }
    println();
}
```



target → 6

1 2 4

0 1 2 6

unique ways

idx

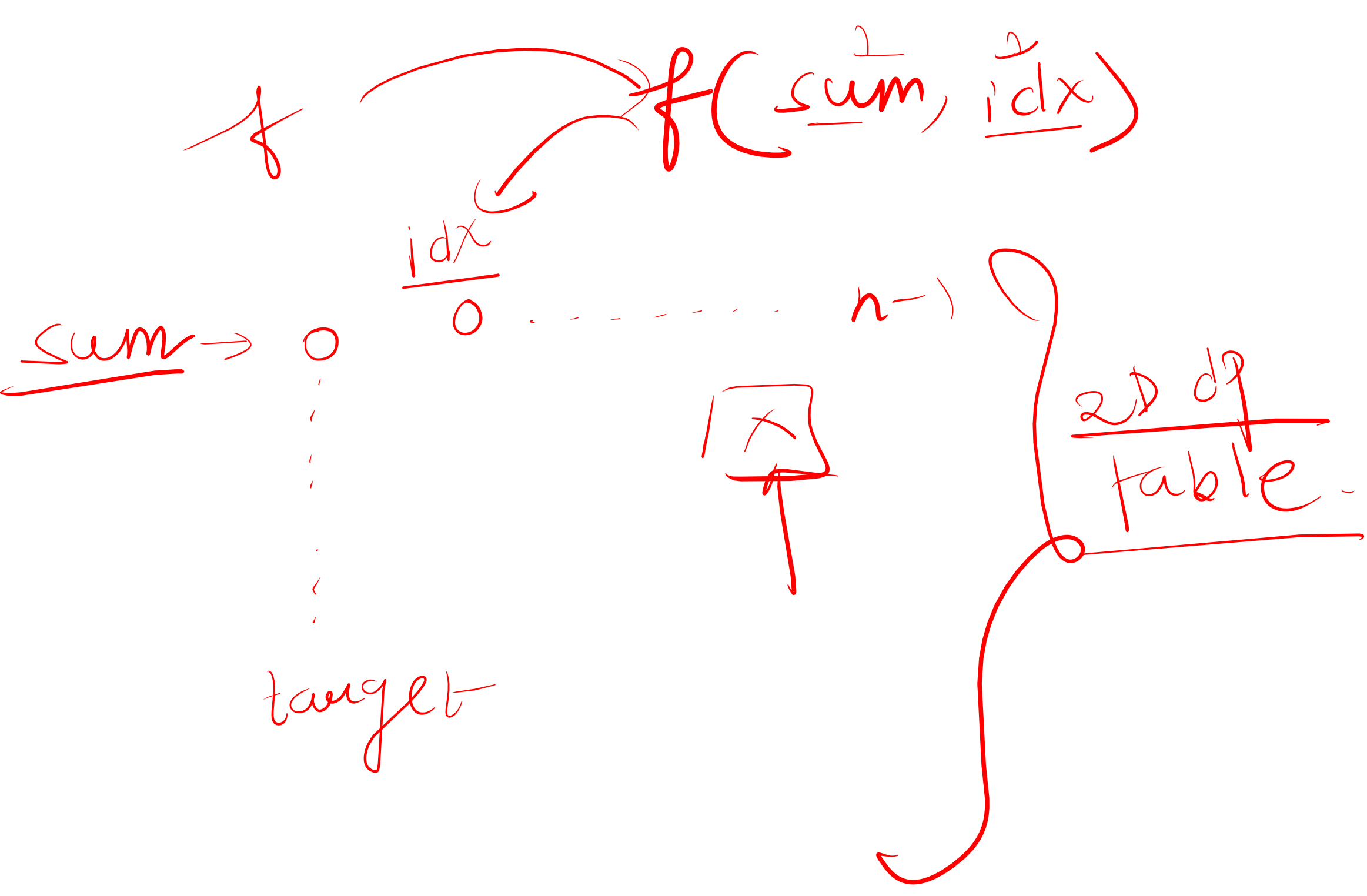
f → sum / target → { 0, 1, 2 }

idx

sum

f → idx, sum

sum


f
f(sum, idx)
idx
0
n-1
sum
target
2D dp
table.

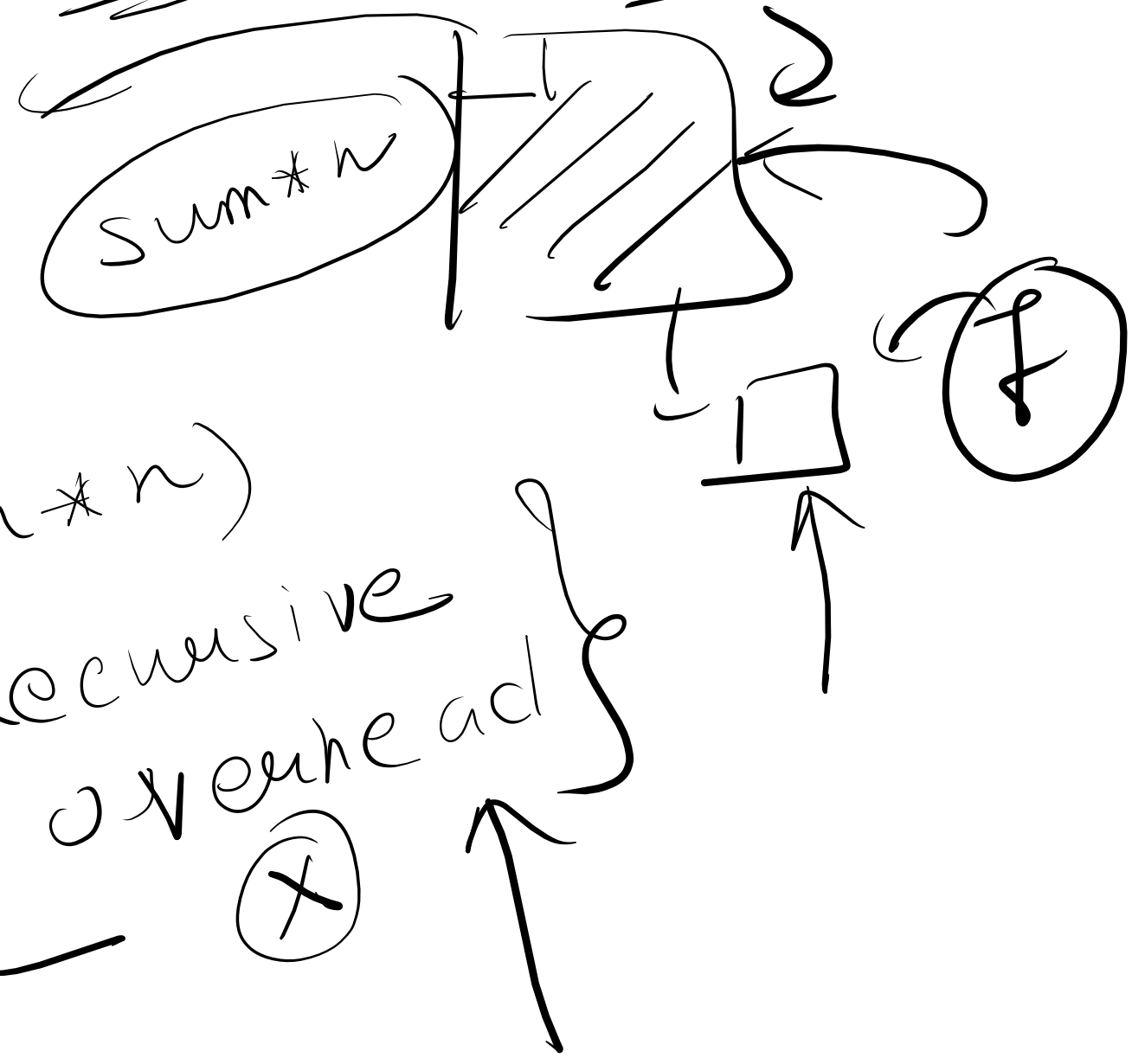
dp[sum+1][idx];

-1

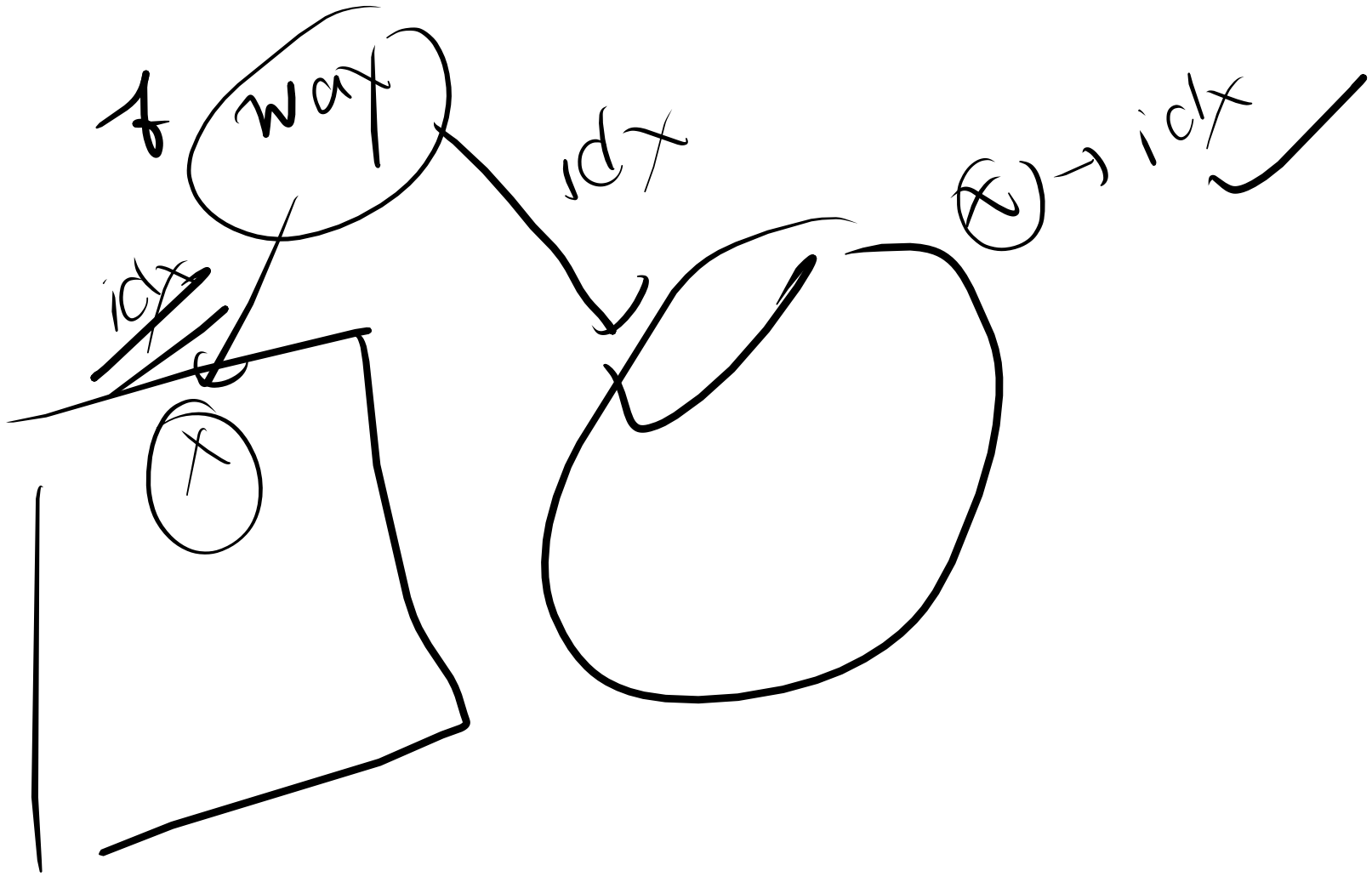
```
f(sum, idx) {
  if(idx == n) {
    if(sum == 0)
      return 1;
    return 0;
  }
  if(dp[sum][idx] != -1)
    return dp[sum][idx];
  w1 = f(sum, idx+1);
  w2 = 0;
  if(coins[idx] <= sum)
    w2 = f(sum-coin[idx], idx);
  return dp[sum][idx] = w1+w2;
}
```

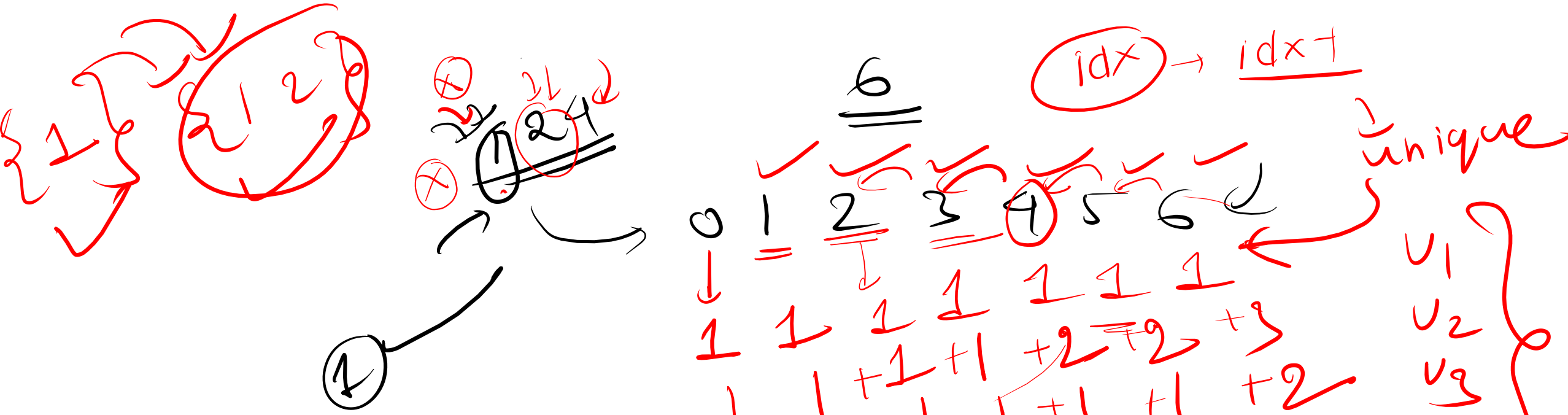
TC :

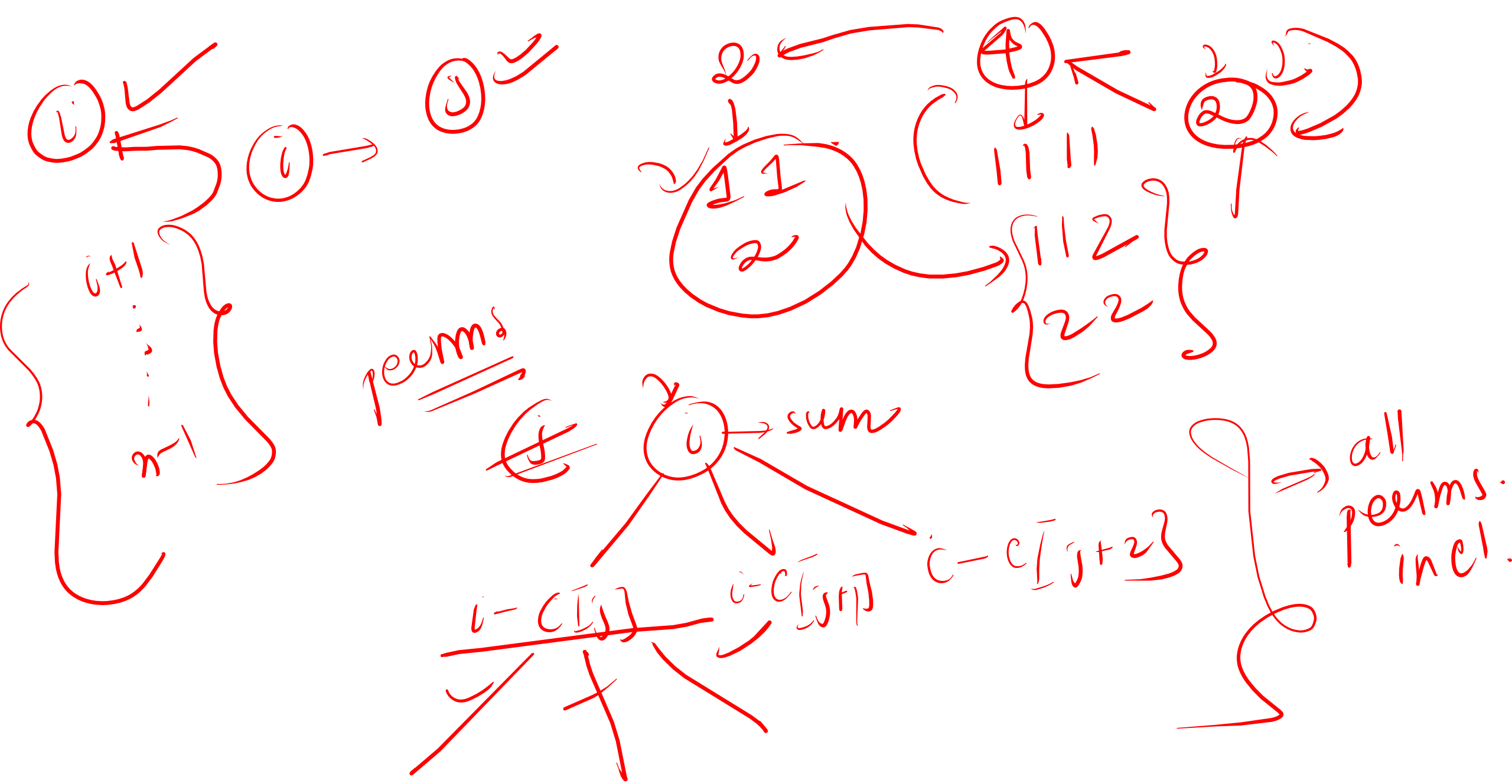
sum * n



1D DP ⇒ thought process




idx → idx+1
unique
v1
v2
v3
ci
y
y - ci


i
i
i+1
n-1
perms
sum
i - c[j]
i - c[j+1]
i - c[j+2]
all
perms.
incl.

$O(target * n)$ ⟹ for(i=0; i<n; i++){

for(j=1; j<=target; j++){

if(c[i]<=j)

dp[j] += dp[j-c[i]);

}

}

⟹ $O(target)$

1D ⟹ sc

pick not pick

sum → idx

sum

sum/2

true

0 1 2 3 6 ⇒ {1 2 3} {6}

0 6

0 1 2 3 4 5 6

True True

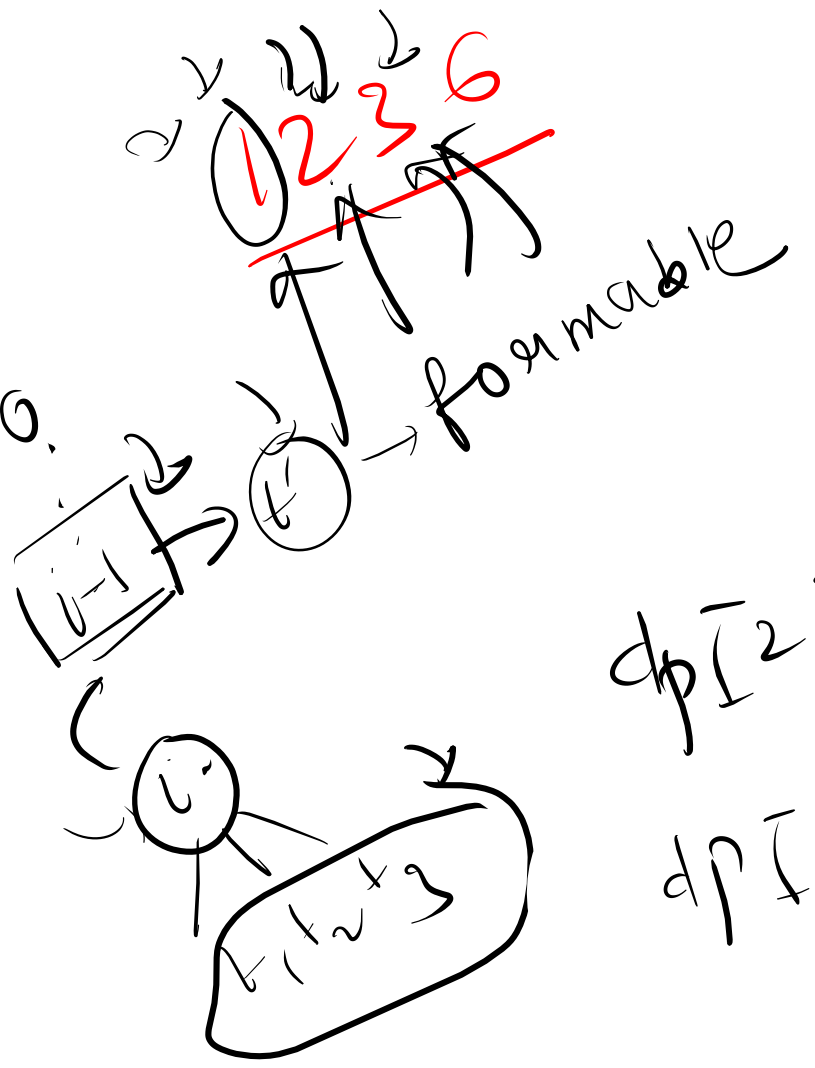
2 - 1 = 1

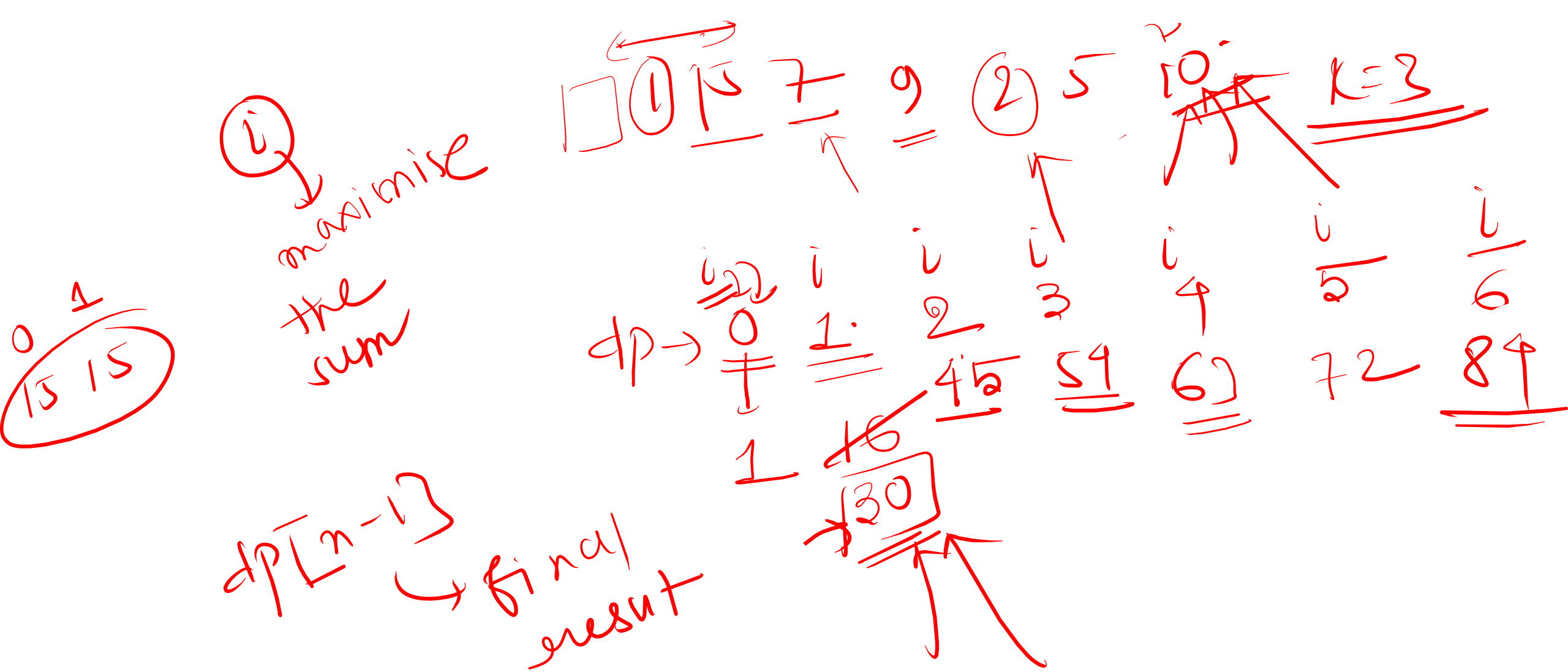
{2 1} {1}

sum → idx

sum :

elements \ idx	0	1	2	3	4	5	6
0	T	T	F	F	F	F	F
1	T	F	T	T	F	F	F
2	T	F	T	T	T	T	T
3	T						F




maximise the sum
k=3
dp
dp[n-1] → final result

ASCII

"aA126*%"

String

"abcde"

String

String → Java

string → Raw data

data → data type

class { fields/property

↳ Primitive data type ⊗

↳ class

↳ field → data

store data

f1 f2 f3

object → vars → "abc"

f1 f2 f3

data

data type

objects

obj

ref

var

"abc"

object of class string

abc

var.f1

var.f2

"abc"

count

int a = 4

a.add(4)

class

lib

String s = "abc";

literals

class

object

RAM

stack

heap

s → "abc"

any object

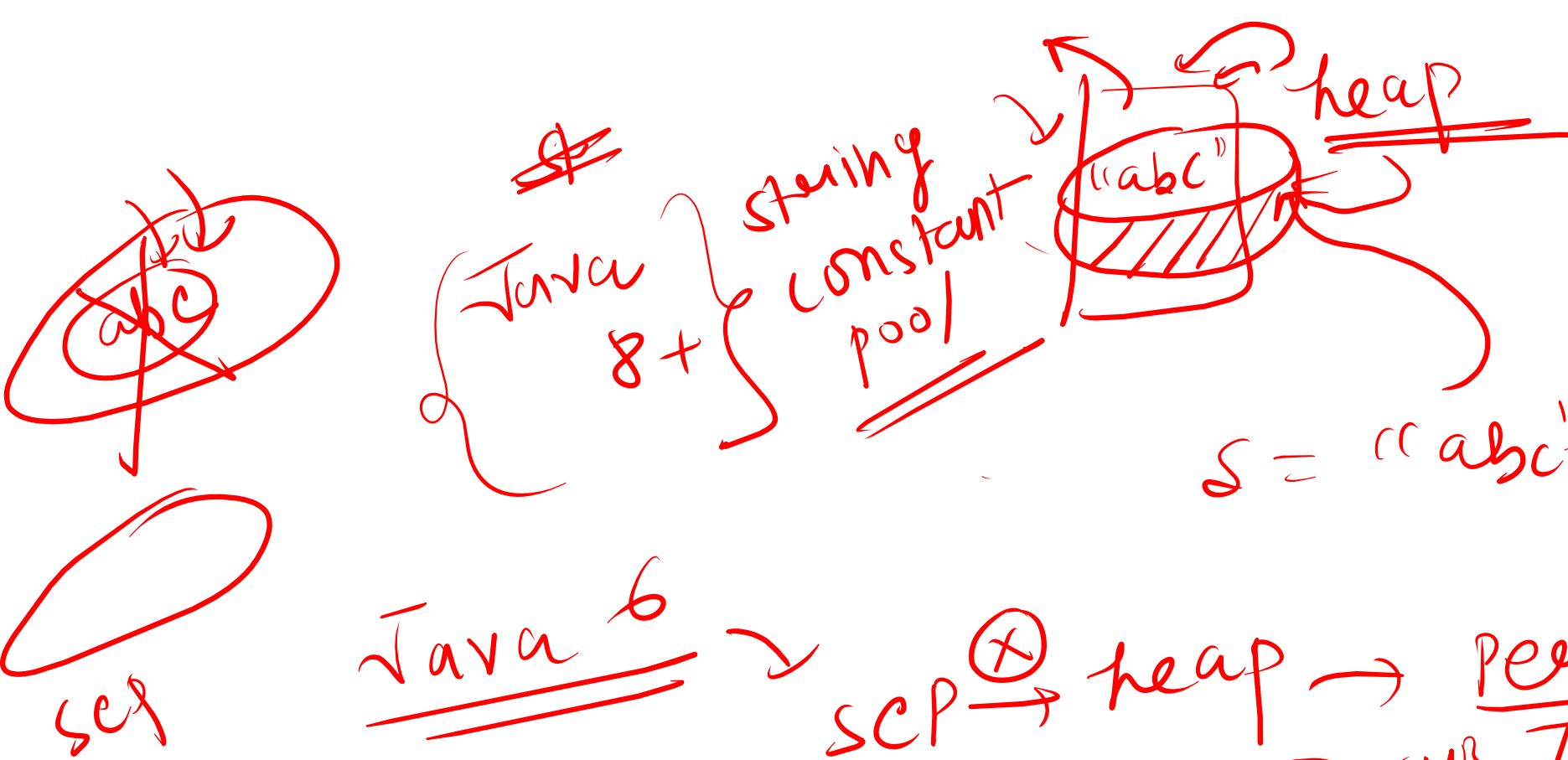
print(s)

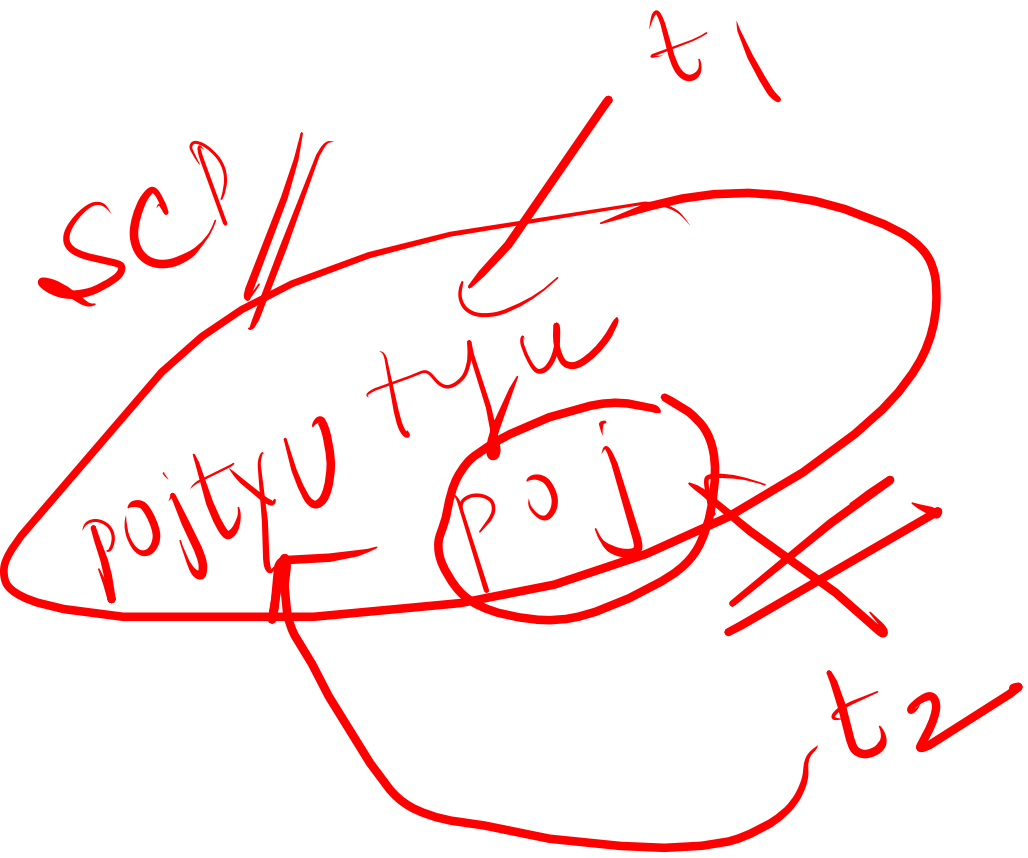
abc

Console

runtime

JVM

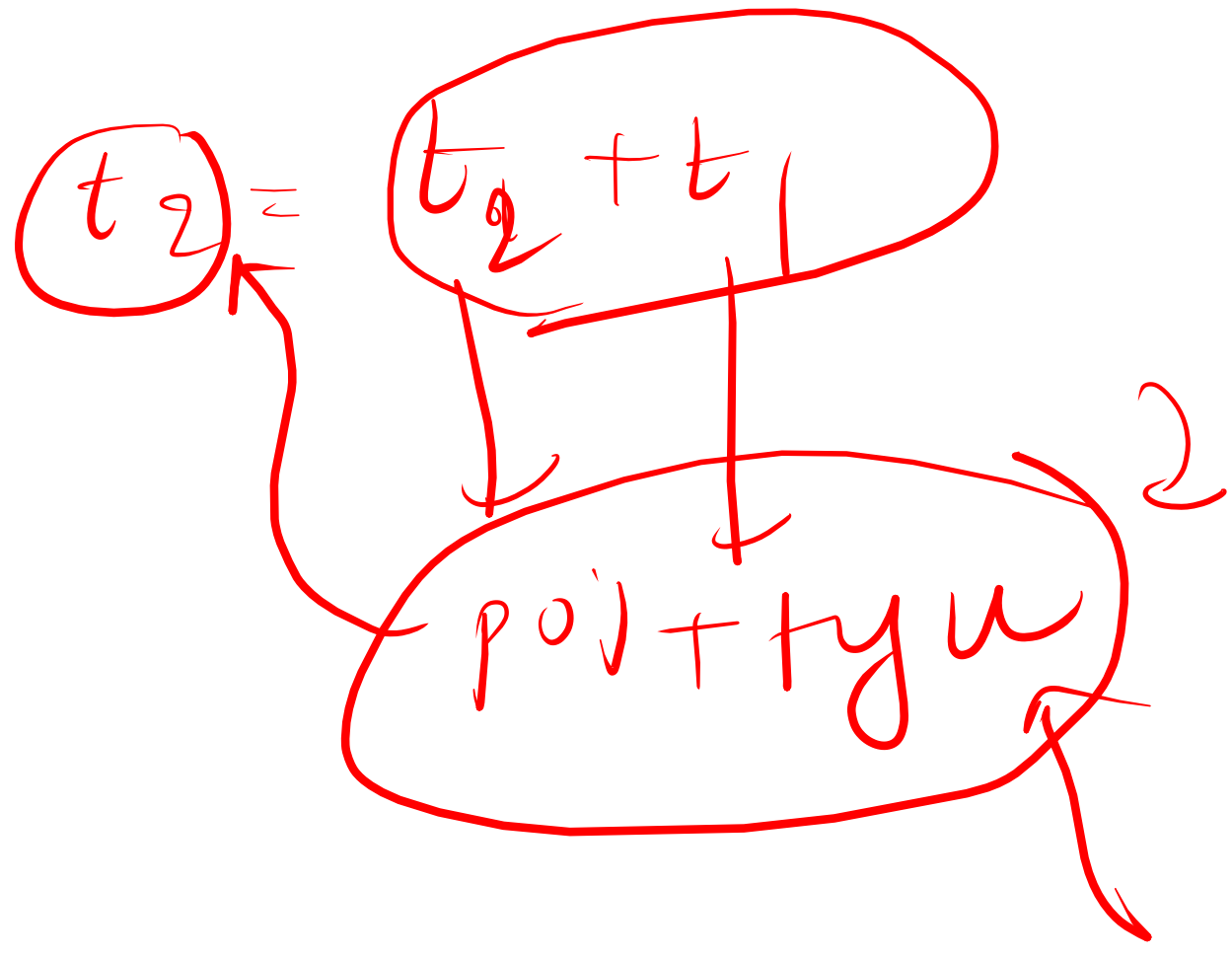

abc
scp
Java 8+
string constant pool
heap
"abc"
s = "abc"
Java 6
scp
heap
permGen
manually develop
fixed
256MB
SCP



$t_1 = tyuu$

$t_2 = poj$

} scp



①

abc

a → (0,0)

ab → (0,1)

abc → (0,2)

b → (1,1)

b → (1,2)

c → (2,2)

abcde

substring ?

i i+1 i+2 ... j

substring

n = 3

$\frac{n(n+1)}{2}$

$\frac{3(4)}{2}$

6

1 1 ... 1

substring

$abcd = n = 4$

$0 \rightarrow a$

$\rightarrow ab$

$\rightarrow abc$

$\rightarrow abcd$ $\} \rightarrow n$

$1 \rightarrow b$

bc

bcd $\} \rightarrow n-1$

$2 \rightarrow c$

cd $\} \; 2$ $(n-2)$

$3 \rightarrow d$ $\} \; 1$ $(n-1)$

$n + n-1 + n-2 \cdots + 2$

$n-1 \rightarrow 1$

n

$$\frac{n(n+1)}{2}$$

$$\frac{4(5)}{2} = 10$$